МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №1
по курсу “Компьютерная графика”

Основы 3D графики

Выполнил: Н. А. Отмахов
Группа: М8О-311БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы. Познакомиться с построением простых 3D-объектов, проекцией на 2D-плоскость, а также научиться работать с матрицами перспективы, ортографической проекции и аффинными преобразованиями.

Вариант 6: Пирамида (правильный тетраэдр).

Требования. Использовать Vulkan, GLFW и ImGui. Программа должна работать в реальном времени, с возможностью динамической смены проекции и трансформаций объектов, корректно отрисовывать объекты с учётом проекции и позволять взаимодействие с пользователем.

Дополнительные задания:

1. Переключение матрицы проекции (ортографическая/перспективная) в интерфейсе.
2. Элементы интерфейса для изменения позиции, поворота и растяжения фигуры.
3. Движение и поворот фигуры по сложной траектории, пауза/воспроизведение анимации, изменение скорости и других параметров траектории.
4. Изменение цвета фигуры через ColorEdit.
5. Процедурные цвета для каждой вершины, умножение на выбранный цвет.
6. Отрисовка нескольких объектов через несколько наборов дескрипторов.

Метод решения

Правильный тетраэдр строится по координатам, вписанным в сферу радиуса r = 1:

- v0 = (0, r, 0) — верхняя вершина,
- три нижние вершины лежат в плоскости y = -r/3 и образуют равносторонний треугольник радиуса $2\sqrt{2}/3 \cdot r$.

Для работы с матрицами используется библиотека GLM.

Каждый объект имеет свой Uniform Buffer Object (UBO) со следующей структурой:

```
struct UniformBufferObject {
   alignas(16) glm::mat4 model;
   alignas(16) glm::mat4 view;
   alignas(16) glm::mat4 proj;
   alignas(16) glm::vec4 baseColor;   // .a = alpha
};
```

view и proj одинаковы для всех объектов; model и baseColor — свои.

В вершинном шейдере цвет вычисляется от локальной позиции вершины:

```
vec3 procedural = inPosition * 0.5 + 0.5;   // [-1, 1] -> [0, 1]
fragColor = procedural * inColor * ubo.baseColor.rgb;
```

Это даёт градиент по каждой грани + позволяет пользователю «подкрашивать» фигуру через baseColor.

Для корректной полупрозрачности:

Включён alpha blending: src * srcAlpha + dst * (1 - srcAlpha).

depthWriteEnable = VK_FALSE — чтобы задние грани не блокировались depth-буфером.

Painter's algorithm: и грани внутри объекта, и сами объекты сортируются по расстоянию от камеры (дальние рисуются первыми).

Для задания 6 создан массив GameObject, для каждого:

- свой VkBuffer под UBO,
- свой VkDescriptorSet (аллоцируется из общего пула размером MAX_OBJECTS),
- свой набор трансформаций и параметров анимации.

При отрисовке циклически вызывается vkCmdBindDescriptorSets(...&obj.descriptorSet...) перед vkCmdDrawIndexed. Pipeline один и тот же — переключаются только данные.

Результаты

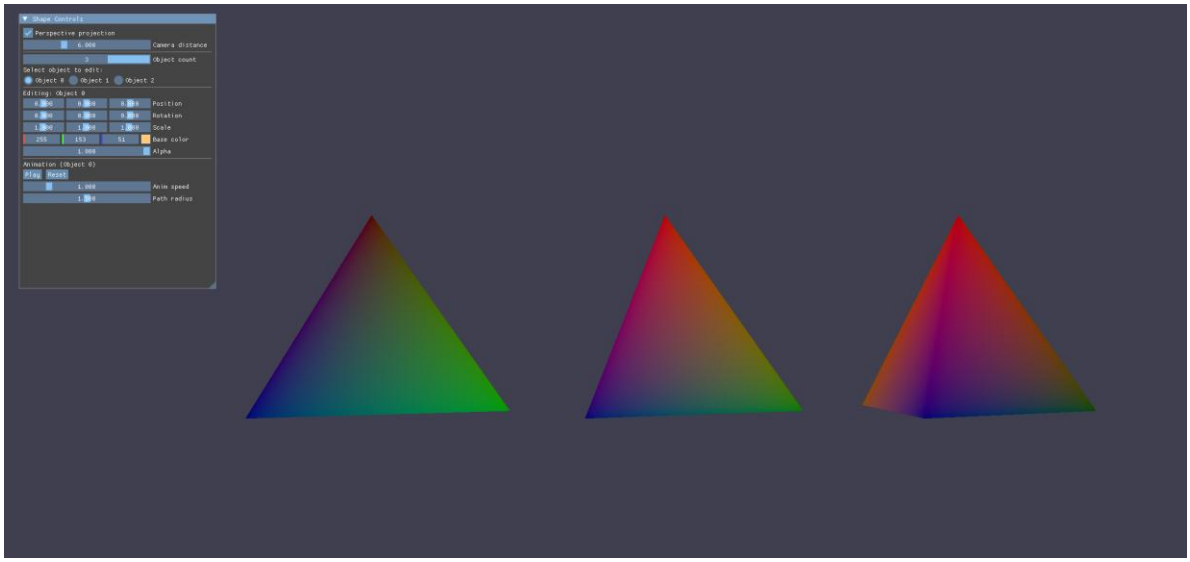


Рис. 1 – Три пирамиды в перспективной проекции



Рис. 2 – Три пирамиды в ортографической проекции
(полупрозрачность и анимация)

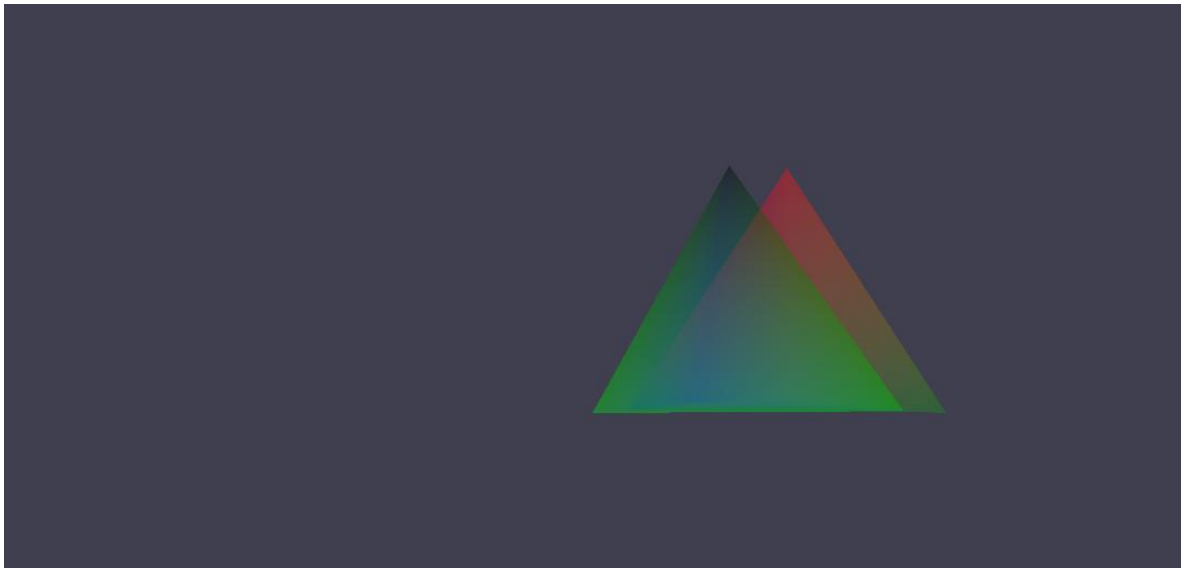


Рис. 3 – Траектории пирамид пересекаютс

Выводы

В ходе лабораторной работы я освоил базовый пайплайн Vulkan, научился строить 3D-объекты, изучил разницу между перспективной и ортографической проекциями, реализовал полупрозрачность, сделал процедурные цвета вершин.